ContextWorld: Benchmarking In-Context Dynamics Adaptation in Latent Visual World Models

Abstract

Visual world models learn environment dynamics, which describe how states change in response to actions, to predict future states. Latent world models encode visual observations as latent states and predict their evolution. Under different dynamics, the same current observation and action sequence can lead to different outcomes. Interaction history can provide evidence about the environment's hidden dynamics. However, a model can achieve low prediction error without using interaction history. We therefore introduce ContextWorld, a benchmark for in-context dynamics adaptation. We replace the interaction history while holding the current observation and future actions fixed. We then measure prediction changes and evaluate prediction accuracy. The benchmark includes nine tasks across four environments and provides separate training and evaluation data. Evaluation data come from continuous interactions and include paired samples with histories and ground-truth future states under different dynamics. With weights fixed, models receive only a history of visual observations and executed actions, together with a future action sequence. Our evaluation of widely used latent world models shows that models trained only under fixed dynamics make limited use of interaction history. Mixing ContextWorld training data with the original data improves this ability, with gains from data scaling varying across tasks. Different models excel on different tasks, and effective training strategies vary across models and tasks. ContextWorld reveals these differences in history use and prediction accuracy. Further analysis shows that, under some training strategies, models make limited use of history even on the training set, despite decreasing training loss. This suggests that existing training objectives may not sufficiently encourage in-context dynamics adaptation.

1. Introduction

Visual world models learn environment dynamics and predict future states [17]. Latent world models make these predictions in a learned representation of visual observations [18, 19]. A model trained under fixed dynamics may predict accurately without using interaction history to adapt its predictions to the current dynamics. Such predictions may fail when the dynamics change [30]. For example, visually identical objects can move different distances under the same push if friction differs. Interaction history can provide evidence about these hidden conditions. We ask whether models can use this evidence to predict future states under the current dynamics without updating their weights, an ability we call *in-context dynamics adaptation*.

Physical prediction benchmarks test whether observed interactions help models predict outcomes governed by hidden properties [4, 29]. We introduce ContextWorld to evaluate this ability directly in action-conditioned latent world models, using their existing predictors without training a separate classifier for future events. The benchmark covers nine tasks across four environments and provides separate training and evaluation data. In its paired tasks, two dynamics conditions share the same current observation and future actions but produce different true futures. Interaction history is therefore necessary to distinguish the two outcomes. We first measure the error between each prediction and its corresponding true future. When the dynamics change, the same actions can lead to different outcomes. We test whether the model predicts these differences accurately. To test whether interaction history helps prediction, we replace it with history from the other dynamics setting. In this control, we retain the original future as the scoring target: an increase in error indicates that the original history helped predict this future. Prediction errors are measured within each model's latent space. Histories and true futures come from continuous simulator interactions; models receive only visual observations and actions, and their weights remain fixed during evaluation. The training data allow us to examine whether exposure to varied dynamics helps models acquire this ability.

We evaluate LeWM [22], PLDM [27], and DINO-WM [34] and find limited use of interaction history on most tasks after training under fixed dynamics. To examine how this ability can be learned, we mix ContextWorld training data with the original data and compare training from scratch with fine-tuning models pretrained under fixed dynamics. We train models separately for each task; DINO-

WM retains its frozen pretrained visual encoder in both settings. We scale up the ContextWorld training data and examine how freezing the visual encoder during fine-tuning affects the use of interaction history in LeWM and PLDM. Data scaling improves history use on several tasks, while the most effective model and training strategy vary by task. For example, starting from the same pretrained weights, freezing PLDM's visual encoder improves its ability to distinguish future states when contact friction varies, but reduces this ability when motion damping varies.

To examine why some models make limited use of history, we compare their predictions on training and held-out data. Under some training strategies, training loss decreases while models still make limited use of interaction history, even on the training data. These failures cannot be explained by generalization alone and suggest that the training objectives may not sufficiently encourage models to use interaction history for prediction.

Our contributions are:

1. **The ContextWorld benchmark.** We introduce ContextWorld, a benchmark with training and evaluation data for nine hidden-dynamics tasks across four environments. It measures history use and prediction accuracy directly from action-conditioned latent predictions, with model weights fixed and no additional learned readout.
2. **A systematic comparison of models and training strategies.** We compare three latent world models across data scales and training strategies. Data scaling improves history use on several tasks, while the most effective model and training strategy vary by task.
3. **Evidence of adaptation failures during training.** We find that, under some training strategies, training loss decreases while models still make limited use of interaction history on the training data. These failures cannot be explained by generalization alone.

2. Related Work

Visual world models. World models learn environment dynamics to predict future states. The World Models framework [17] combines a learned visual representation with recurrent dynamics to support policy training inside the learned model. PlaNet [18] uses learned latent dynamics to select actions through online planning. Dreamer [19] instead learns policies from imagined latent trajectories. These methods use image reconstruction to learn their visual representations. Joint-embedding predictive architectures (JEPAs) [21] offer a different training objective: predicting target representations without reconstructing pixels. I-JEPA [2] applies this principle to masked image regions, and V-JEPA [5] extends it to video representation learning. To support robotic planning, V-JEPA 2 [3] adds an action-conditioned predictor trained on robot interactions using pretrained video representations. Action-conditioned world models also differ in how they obtain their visual representations. DINO-WM [34] trains a predictor on frozen pretrained features for planning from offline trajectories. PLDM [27] jointly learns the encoder and predictor from reward-free offline data. To stabilize joint training, LeWM [22] simplifies the objective to a latent prediction loss and a single representation regularizer.

In-context adaptation. Environment dynamics can differ between training and deployment. Interaction history can support adaptation without costly retraining. Meta-reinforcement learning [6] trains agents across tasks to adapt from limited experience. RL^2 [14] stores interaction history in a recurrent network's hidden state, while PEARL [26] infers a task representation from experience to condition a policy. Large language models can learn from examples in context without updating their weights [8, 13]. In world models, context can guide predictions of action outcomes in a new environment. OSWM [16] pretrains a transformer on synthetic dynamics and uses observed state transitions as context to predict future states in new environments. L2World [30] distinguishes using context to identify environments learned during training from learning dynamics directly from context. It studies the effects of context length and the diversity of training environments using multi-step prediction errors and context perturbations. For robotic control, ICWM [31] uses task-agnostic interaction history to adapt policies to new camera viewpoints without weight updates. Its ablations compare correct, absent, and mismatched interaction context.

Benchmarks. To evaluate adaptation across environments, existing benchmarks introduce controlled changes in dynamics, appearance, or task rules. Meta-World [32] and Procgen [11] test generalization across manipulation tasks and procedurally generated levels, respectively. CausalWorld [1] and CARL [7] vary environment properties, while XLand-MiniGrid [23] and XLand-100B [24] evaluate adaptation from interaction history across tasks. For physical prediction, CoPhy [4] asks models to predict outcomes after changes to a scene's initial configuration, using the original trajectory as evidence about hidden properties. Filtered-CoPhy [20] requires the observed trajectory to contain enough information to predict the counterfactual outcome. ComPhy [9] tests reasoning about hidden mass and charge through predictive and counterfactual questions about videos. Physion++ [29] tests whether interaction history helps predict future contact in paired trials with the same current observation but different physical properties. Beyond visual prediction, a recent time-series benchmark [33] reports that low prediction error can coexist with incorrect response directions when actions change. A remaining question is when action-conditioned latent world

models learn to use history for accurate prediction. ContextWorld addresses this question through controlled comparisons across models, tasks, data scales, and training strategies.

3. ContextWorld Benchmark

ContextWorld tests whether models use interaction history to accurately predict future states given an action sequence. This section defines the prediction setting, describes how we construct the tasks and datasets, and explains the evaluation protocol and metrics.

3.1. Problem Formulation

To evaluate adaptation through interaction history, we keep model weights θ fixed and provide only visual observations and actions. For each prediction, the model receives an interaction history H_t and a future action sequence $A_{t:t+\tau-1}$. The history contains visual observations up to and including time t, together with the actions executed between them. The prediction and its target are

$$\begin{aligned} \hat{z}_{t+\tau} &= f_\theta(H_t, A_{t:t+\tau-1}), \\ z_{t+\tau} &= e_\theta(o_{t+\tau}). \end{aligned} \tag{1}$$

Here, $o_{t+\tau}$ is the ground-truth observation at time $t+\tau$, and e_θ is the model's target encoder, also fixed during evaluation. Predictions are evaluated against the encoded targets in latent space.

3.2. Task Design

We construct nine tasks by varying the dynamics of four environments. The tasks cover changes in physical parameters and interaction rules. They build on the TwoRoom navigation environment used in DINO-WM [34], the PushT pushing environment [10], Reacher from the DeepMind Control Suite [28], and Cube from OGBench [25]. Table 1 summarizes the nine tasks.

To test whether models use interaction history for prediction, we construct paired samples for six tasks: pusher displacement, arm mass, contact friction, motion damping, gripper carry rule, and portal exit. The paired samples share the current observation and future action sequence, but have different hidden dynamics. For each sample, we generate the history and ground-truth future in one continuous simulation with fixed dynamics. The two futures differ, so accurate prediction requires information from the history. Construction checks are detailed in Appendix B.

The speed task tests whether models can use history to predict motion at unseen speeds. We hold the current observation and future actions fixed and provide histories recorded at different speeds. Action delay covers several delay values to test predictions before and after an action takes effect. For example, with delays of six or ten simulator steps, no movement occurs within the next five steps. These delays produce the same future state, so we score them as one response group and weight all groups equally (Appendix C). For the door task, we test whether the model correctly predicts passage or blockage from the interaction history.

3.3. Dataset

For each task, we provide Training, Development, and Test splits with no shared interaction trajectories. The Development split is used to select methods and checkpoints. These splits may share dynamics parameter values or interaction rules. The speed task also tests speeds not used in training, both within the training range (interpolation) and outside it (extrapolation).

The paired tasks provide 32,768 training pairs per task, except for gripper carry, which provides 10,000. Each has 256 Development pairs and 256 Test pairs. Appendix D provides complete dataset statistics, dynamics settings, and split checks for all nine tasks.

3.4. Evaluation

A model can change its predictions with the interaction history and still predict the wrong future. We therefore evaluate whether predictions distinguish the true futures under different dynamics, measure prediction error, and compare errors with matching and mismatched histories. We report benchmark results on the Test split.

Prediction accuracy. We use task-specific accuracy rules to account for differences in the dynamics conditions and distinguishable future states. For the six paired tasks and door traversability, we compare each prediction with both true futures. A prediction using the history from condition A is counted as correct only if it is closer to A's true future than to B's, and vice versa. For pair i, z_i^A and z_i^B denote the true future representations, and $\hat{z}_i^A$ and $\hat{z}_i^B$ denote the corresponding predictions. Using mean squared error d, we report accuracy as the percentage of correct predictions across both conditions in N pairs:

$$\begin{aligned} A_{\text{future}} &= \frac{100}{2N} \sum_{i=1}^{N} (I_i^A + I_i^B), \\ I_i^A &= \mathbf{1}\{d(\hat{z}_i^A, z_i^A) < d(\hat{z}_i^A, z_i^B)\}, \\ I_i^B &= \mathbf{1}\{d(\hat{z}_i^B, z_i^B) < d(\hat{z}_i^B, z_i^A)\}. \end{aligned} \tag{2}$$

Speed and action delay involve more than two dynamics conditions. For each tested speed, we compare predictions from different histories with the true future at that speed. We count the prediction from the matching history as correct only when its error is lower than that of every other

Table 1. The nine tasks test whether interaction history helps models predict the effects of hidden dynamics. History length h counts observation frames, including the current frame; sampling details are given in Appendix A.

Task	Environment	h	What history helps the model predict
Speed	TwoRoom	3	How far the agent moves under the same action
Pusher displacement	PushT	3	Whether the same action moves the pusher a shorter or longer distance
Arm mass	Reacher	3	How the same torque changes arm motion
Action delay	TwoRoom	7	When an action takes effect
Contact friction	PushT	3	How friction changes object motion during contact
Motion damping	PushT	3	How quickly object motion decays after contact ends
Gripper carry rule	Cube	3	Whether a closed gripper carries the cube
Door traversability	TwoRoom	3	Whether a visually identical door permits passage
Portal exit	TwoRoom	3	Where the agent emerges after entering the same portal

prediction. For action delay, the prediction horizon is five simulator steps. Delays 0–4 form five separate response groups, while delays 5–10 form one group because the action has not yet taken effect. We assign each prediction to the group containing the closest true future, compute accuracy for each group, and report the mean across the six groups. Appendix E gives the detailed scoring rules.

Complete prediction error. A prediction can be closer to the correct future yet still misrepresent position or motion. To measure this error, we give every hidden condition in a scene the same future action sequences. Models roll out from real interaction history without receiving any further ground-truth observations. We compare predictions with true futures at 5, 10, 15, 20, and 25 simulator steps.

We normalize the error by a reference computed from the true futures. For a given scene, action sequence, and prediction time, the mean future minimizes squared error among predictions that are identical across conditions, when those conditions are weighted equally. For scene q, this mean is $\bar{z}_{qct} = K_q^{-1} \sum_k z_{qkct}$. Here, K_q is the number of conditions, C_q is the number of shared action sequences, and $T = 5$ is the number of evaluation times. The true and predicted representations for condition k, action sequence c, and evaluation time t are z_{qkct} and $\hat{z}_{qkct}$. We average over conditions, actions, and times within each scene before aggregating:

$$
\begin{aligned}
L_q &= \frac{1}{K_q C_q T} \sum_{k,c,t} \|\hat{z}_{qkct} - z_{qkct}\|^2, \\
B_q &= \frac{1}{K_q C_q T} \sum_{k,c,t} \|z_{qkct} - \bar{z}_{qct}\|^2, \\
E/B &= \frac{\sum_q L_q}{\sum_q B_q}.
\end{aligned}
\tag{3}
$$

Lower E/B is better: 0 denotes an exact prediction in the target representation, 1 denotes the same error as the mean-future reference, and values above 1 indicate greater error. The reference is used only for scoring and is never provided to the model; the denominator must be positive. When conditions share the current observation and future actions and are equally weighted, a deterministic model that ignores history cannot achieve $E/B < 1$. This ratio measures complete prediction error, but does not establish whether matching history helps more than mismatched history.

History controls. We replace the history with one from a different dynamics condition in the same scene, while keeping the current observation, future actions, and scoring target unchanged. The target remains the true future of the original condition; replacing the model's input does not change the environment being predicted. History (%) reports how often matching history gives strictly lower prediction error than mismatched history. Speed also uses this strict comparison as its task score.

This rate records wins, not the size of the improvement. Let $L_q^+ = L_q$ be the scene error with matching history and L_q^- the error with mismatched history, averaged equally over the alternatives when there is more than one. Using the same weights over conditions, actions, and times, define $E_+ = \sum_q L_q^+$, $E_- = \sum_q L_q^-$, and $B = \sum_q B_q > 0$. The normalized error reduction is

$$
G_H = \frac{E_- - E_+}{B}. \tag{4}
$$

A positive G_H means that matching history reduces error. The remaining prediction error is still measured by E/B.

Conditional response error. Prediction loss can fall as errors shared by both conditions decrease, even if the predicted difference between conditions remains inaccurate. To examine this distinction during training, we compare the true and predicted differences between futures. For a two-condition query, let $\Delta z_i = z_i^B - z_i^A$ and $\Delta \hat{z}_i = \hat{z}_i^B - \hat{z}_i^A$.

The normalized response error (NRE) is

$$\mathrm{NRE} = \frac{\sum_i \|\Delta\hat{z}_i - \Delta z_i\|^2}{\sum_i \|\Delta z_i\|^2}. \tag{5}$$

Lower NRE is better. A value of 0 denotes an exact conditional response; 1 is the error of predicting no difference between conditions. Subtracting predictions removes errors common to both conditions, so NRE cannot replace complete prediction error. Conversely, NRE equal to 1 does not by itself show that a model ignores history. We specify the prediction horizon for each analysis and define NRE only when its denominator is positive.

Interpreting and aggregating results. Task scores measure the distinctions specified by each task. Both E/B and NRE are computed in each checkpoint's own target representation and do not provide a common measure of physical accuracy for comparing models. If the target encoder changes during training, comparisons across checkpoints also reflect changes in the representation. Training analyses therefore describe each checkpoint in its own target space; error comparisons are more direct when that space is fixed. The metrics describe different aspects of the same predictions and are not independent pieces of evidence.

We weight training runs with the same settings equally and report their standard deviation. For multi-step errors, bootstrap resampling [15] groups samples by source trajectory or scene, retaining related samples together. These intervals describe scene-sampling uncertainty for the evaluated checkpoints. We report planning performance with the cross-entropy method (CEM) [12] in the original environments separately to measure retention of planning ability. Since these environments need not require adaptation to changing dynamics, their planning scores do not replace the adaptation evaluation.

4. Experiments

4.1. Experimental Setup

For baseline training and mixed-data training, we use the original-environment datasets released with LeWM [22]. For mixed-data training, we sample equal proportions of trajectory windows from these datasets and the corresponding task's ContextWorld Training split.

References

[1] Ossama Ahmed, Frederik Träuble, Anirudh Goyal, Alexander Neitz, Yoshua Bengio, Bernhard Schölkopf, Manuel Wüthrich, and Stefan Bauer. CausalWorld: A Robotic Manipulation Benchmark for Causal Structure and Transfer Learning. In *International Conference on Learning Representations*, 2021. 2

[2] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-Supervised Learning From Images With a Joint-Embedding Predictive Architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15619–15629, 2023. 2

[3] Mahmoud Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning. *arXiv preprint arXiv:2506.09985*, 2025. 2

[4] Fabien Baradel, Natalia Neverova, Julien Mille, Greg Mori, and Christian Wolf. CoPhy: Counterfactual Learning of Physical Dynamics. In *International Conference on Learning Representations*, 2020. 1, 2

[5] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting Feature Prediction for Learning Visual Representations from Video. *arXiv preprint arXiv:2404.08471*, 2024. 2

[6] Jacob Beck, Risto Vuorio, Evan Zheran Liu, Zheng Xiong, Luisa Zintgraf, Chelsea Finn, and Shimon Whiteson. A Tutorial on Meta-Reinforcement Learning. *Foundations and Trends in Machine Learning*, 18(2–3):224–384, 2025. 2

[7] Carolin Benjamins, Theresa Eimer, Frederik Schubert, Aditya Mohan, Sebastian Döhler, André Biedenkapp, Bodo Rosenhahn, Frank Hutter, and Marius Lindauer. Contextualize Me – The Case for Context in Reinforcement Learning. *Transactions on Machine Learning Research*, 2023. 2

[8] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, 2020. 2

[9] Zhenfang Chen, Kexin Yi, Yunzhu Li, Mingyu Ding, Antonio Torralba, Joshua B. Tenenbaum, and Chuang Gan. ComPhy: Compositional Physical Reasoning of Objects and Events from Videos. In *International Conference on Learning Representations*, 2022. 2

[10] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin CM Burchfiel, and Shuran Song. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. 3

[11] Karl Cobbe, Chris Hesse, Jacob Hilton, and John Schulman. Leveraging Procedural Generation to Benchmark Reinforce-

ment Learning. In *Proceedings of the 37th International Conference on Machine Learning*, pages 2048–2056. PMLR, 2020. 2

[12] Pieter-Tjerk de Boer, Dirk P. Kroese, Shie Mannor, and Reuven Y. Rubinstein. A Tutorial on the Cross-Entropy Method. *Annals of Operations Research*, 134(1):19–67, 2005. 5

[13] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Jingyuan Ma, Rui Li, Heming Xia, Jingjing Xu, Zhiyong Wu, Baobao Chang, Xu Sun, Lei Li, and Zhifang Sui. A Survey on In-context Learning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 1107–1128. Association for Computational Linguistics, 2024. 2

[14] Yan Duan, John Schulman, Xi Chen, Peter L. Bartlett, Ilya Sutskever, and Pieter Abbeel. RL^2: Fast Reinforcement Learning via Slow Reinforcement Learning. *arXiv preprint arXiv:1611.02779*, 2016. 2

[15] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993. 5

[16] Fabio Ferreira, Moreno Schlageter, Raghu Rajan, André Biedenkapp, and Frank Hutter. One-shot World Models Using a Transformer Trained on a Synthetic Prior. *arXiv preprint arXiv:2409.14084*, 2024. 2

[17] David Ha and Jürgen Schmidhuber. Recurrent World Models Facilitate Policy Evolution. In *Advances in Neural Information Processing Systems*, 2018. 1, 2

[18] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning Latent Dynamics for Planning from Pixels. In *Proceedings of the 36th International Conference on Machine Learning*, pages 2555–2565. PMLR, 2019. 1, 2

[19] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to Control: Learning Behaviors by Latent Imagination. In *International Conference on Learning Representations*, 2020. 1, 2

[20] Steeven Janny, Fabien Baradel, Natalia Neverova, Madiha Nadri, Greg Mori, and Christian Wolf. Filtered-CoPhy: Unsupervised Learning of Counterfactual Physics in Pixel Space. In *International Conference on Learning Representations*, 2022. 2

[21] Yann LeCun. A Path Towards Autonomous Machine Intelligence, 2022. Position paper. 2

[22] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorldModel: Stable End-to-End Joint-Embedding Predictive Architecture from Pixels. *arXiv preprint arXiv:2603.19312*, 2026. 1, 2, 5

[23] Alexander Nikulin, Vladislav Kurenkov, Ilya Zisman, Artem Agarkov, Viacheslav Sinii, and Sergey Kolesnikov. XLand-MiniGrid: Scalable Meta-Reinforcement Learning Environments in JAX. In *Advances in Neural Information Processing Systems*, 2024. 2

[24] Alexander Nikulin, Ilya Zisman, Alexey Zemtsov, and Vladislav Kurenkov. XLand-100B: A Large-Scale Multi-Task Dataset for In-Context Reinforcement Learning. In *International Conference on Learning Representations*, 2025. 2

[25] Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. OGBench: Benchmarking Offline Goal-Conditioned RL. In *International Conference on Learning Representations*, 2025. 3

[26] Kate Rakelly, Aurick Zhou, Chelsea Finn, Sergey Levine, and Deirdre Quillen. Efficient Off-Policy Meta-Reinforcement Learning via Probabilistic Context Variables. In *Proceedings of the 36th International Conference on Machine Learning*, pages 5331–5340. PMLR, 2019. 2

[27] Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Learning from Reward-Free Offline Data: A Case for Planning with Latent Dynamics Models. *arXiv preprint arXiv:2502.14819*, 2025. 1, 2

[28] Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, Timothy Lillicrap, and Martin Riedmiller. DeepMind Control Suite. *arXiv preprint arXiv:1801.00690*, 2018. 3

[29] Hsiao-Yu Tung, Mingyu Ding, Zhenfang Chen, Daniel M. Bear, Chuang Gan, Joshua B. Tenenbaum, Daniel L. K. Yamins, Judith Fan, and Kevin A. Smith. Physion++: Evaluating Physical Scene Understanding that Requires Online Inference of Different Physical Properties. In *Advances in Neural Information Processing Systems*, 2023. 1, 2

[30] Fan Wang, Zhiyuan Chen, Yuxuan Zhong, Sunjian Zheng, Pengtao Shao, Bo Yu, Shaoshan Liu, Jianan Wang, Ning Ding, Yang Cao, and Yu Kang. Context and Diversity Matter: The Emergence of In-Context Learning in World Models. In *International Conference on Learning Representations*, 2026. 1, 2

[31] Siyin Wang, Junhao Shi, Senyu Fei, Zhaoyang Fu, Li Ji, Jingjing Gong, and Xipeng Qiu. In-Context World Modeling for Robotic Control. *arXiv preprint arXiv:2606.26025*, 2026. 2

[32] Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-World: A Benchmark and Evaluation for Multi-Task and Meta Reinforcement Learning. In *Proceedings of the Conference on Robot Learning*, pages 1094–1100. PMLR, 2020. 2

[33] Haochen Zhang, Jiaheng Guo, Zhen Xu, Zachary Plotkin, Nicholas Konz, Zhen Tan, and Tianlong Chen. On the Divergence of Accuracy and Mechanism Consistency in Time Series World Models. *arXiv preprint arXiv:2610.01842*, 2026. 2

[34] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World Models on Pre-trained Visual Features enable Zero-shot Planning. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 79115–79135. PMLR, 2025. 1, 2, 3

A. Input Sampling

Observations are typically sampled every five simulator steps. The actions executed between consecutive observations are included in the interaction history.

B. Trajectory Construction and Pairing

Continuous trajectories. Each sample follows a continuous simulator trajectory from its history through its future, without intermediate resets or state overrides. Identical current observations may still conceal differences in velocity or contact state, so we perform task-specific checks of physical-state matching at the start of prediction. For action delay, we retain the actual queue of pending actions.

Starting positions. For tasks in which the two dynamics conditions cannot naturally share a starting position, we exchange their starting positions during generation and verify that the two conditions have identical sets of starting images. This check reduces the possibility that the starting image alone reveals the condition.

Predictions without history. Consider a complete, balanced pair with distinguishable encoded targets. A deterministic model that ignores history receives the same current observation and future action sequence for both samples and therefore produces the same prediction. This prediction cannot be strictly closer to each sample's own target than to the other target. Consequently, such a model scores at most 50% on the paired future-selection metric in Eq. 2; ties count as failures.

C. Action Delay Response Groups

The delay data retain all 11 delays from 0 to 10 simulator steps. At the five-step horizon used for the main one-step score, delays of 5–10 produce the same stationary response and are grouped together. This score therefore uses six distinguishable response groups: 0, 1, 2, 3, 4, and 5–10.

D. Dataset Statistics and Splits

D.1. Dataset Sizes

Table 2 lists the task data used in the training and evaluation comparisons. Training sizes exclude the original-environment data used for mixed training. A pair contains two trajectories with different dynamics conditions; a trajectory count refers to recorded episodes, not to the number of training windows sampled from them. An evaluation case fixes the current observation and future action sequence and can include histories and targets from several dynamics conditions.

The action delay counts refer to the dataset covering all delays from 0 to 10 simulator steps. Its Training split contains 5,120 base cases, each simulated under all 11 delays, giving 56,320 trajectories. Development and Test each contain 300 base cases, again evaluated under all 11 delays. Door Training contains 7,680 trajectories for each of the passable and blocked rules.

Several examples can share a source episode: pusher displacement can use multiple action windows from one episode, and forward and mirrored damping examples can share a source. The 10,000 gripper carry training pairs use 10,000 distinct source episodes.

D.2. Speed Values and Door Positions

Table 3 lists the speed values used for Training and evaluation. Development and Test use the same evaluation speeds but different interaction trajectories. Each split contains 300 cases for each of the 14 evaluated speeds, or 4,200 cases in total before comparing histories from different speeds. The main task results use the unseen speeds within the training range.

Door position is specified by an integer coordinate along the wall. Test uses the 42 positions $30 + 4k$, where $k = 0, \ldots, 41$. Development uses the 16 positions 65, 77, 79, 87, 112, 115, 131, 152, 167, 169, 171, 172, 175, 180, 183, 185. Training uses the integers from 36 through 188 after excluding all Development and Test positions and the additional unused positions 40, 127, and 132, leaving 96 positions. The three position sets are disjoint. Both passable and blocked rules are used across the splits; the test distinction is the door position, not a new rule.

D.3. Split Checks and Model Inputs

Split checks use source episode identifiers, trajectory template identifiers, or observation hashes, as appropriate for each task. Speed Development uses independently generated initial configurations and excludes initial states and current observations that duplicate Test cases. For Door, we also check the position sets listed above for overlap.

Hidden dynamics values, simulator states, and pair identifiers are retained for data generation, validation, and scoring. They are not passed to the prediction model, which receives only the visual observations and actions defined in Sec. 3.1.

E. Prediction Accuracy Details

Paired tasks. The strict comparisons in Eq. 2 do not count ties as correct. The two comparisons within a pair are dependent. Appendix B explains the upper bound for a deterministic model that ignores history.

Speed. For each target speed, accuracy is the proportion of cases in which matching history gives strictly lower pre-

Table 2. Dataset sizes and condition coverage.

Task	Training	Development	Test
Speed	16,384 trajectories; 32 speeds	300 cases per speed	300 cases per speed
Pusher displacement	32,768 pairs	256 pairs	256 pairs
Arm mass	32,768 pairs	256 pairs	256 pairs
Action delay	56,320 trajectories; 11 delays	300 cases $\times$ 11 delays	300 cases $\times$ 11 delays
Contact friction	32,768 pairs	256 pairs	256 pairs
Motion damping	32,768 pairs	256 pairs	256 pairs
Gripper carry rule	10,000 pairs	256 pairs	256 pairs
Door traversability	15,360 trajectories; 96 door positions	300 cases; 16 door positions	300 cases; 42 door positions
Portal exit	32,768 pairs	256 pairs	256 pairs

Table 3. Speed values in the benchmark. The four evaluation groups are used in both Development and Test. Interpolation values lie within the Training range but are absent from Training.

Setting	Speed values
Training	2.6, 2.7, 2.8, 2.9, 3.1, 3.3, 3.5, 3.7, 3.8, 3.9, 4.1, 4.2, 4.4, 4.5, 4.7, 5.0, 5.1, 5.3, 5.5, 5.7, 5.9, 6.1, 6.2, 6.3, 6.6, 6.7, 6.8, 7.0, 7.2, 7.3, 7.8, 7.9
Seen speeds	3.1, 5.1, 7.0
Interpolation	3.4, 4.8, 6.9
Lower extrapolation	1.75, 1.95, 2.15, 2.35
Upper extrapolation	8.25, 8.75, 9.5, 10.25

diction error than every alternative history. We then average accuracy equally across target speeds. Ties do not count as correct: if all histories produce the same prediction, accuracy is zero. The main results use the interpolation speeds listed in Appendix D.

Action delay. We select the candidate true future with the lowest mean squared error, resolving ties in favor of the smaller delay. The prediction is correct when the selected delay and the true delay belong to the same response group. Within each case, we first average correctness over delays in each group, then average equally across the six groups defined in Appendix C. The final accuracy is the mean over cases.

Door traversability. For a prediction made with matching history, we compare the mean squared errors to the passage and blockage futures. The prediction is correct only if the error to the true future is strictly smaller than the error to the alternative future. Accuracy is averaged over cases and the two rules; ties do not count as correct.